

LiDAR Tracking

LiDAR Tracking

1. Course Content
2. Demo
3. Source Code Analysis

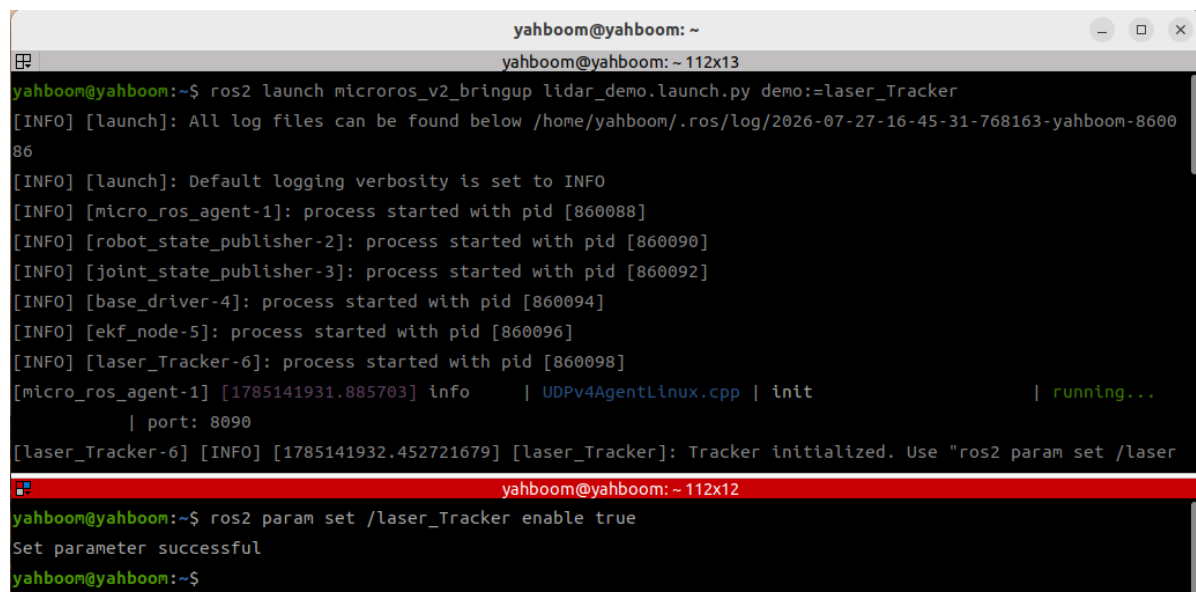
1. Course Content

- Start the demo program: the robot uses the LiDAR to detect the closest object in the surrounding environment and follow it.

2. Demo

```
ros2 launch microros_v2_bringup lidar_demo.launch.py demo:=laser_Tracker
```

- After startup, the program stays silent by default and does not move the chassis



```
yahboom@yahboom: ~  
yahboom@yahboom: ~ 112x13  
yahboom@yahboom:~$ ros2 launch microros_v2_bringup lidar_demo.launch.py demo:=laser_Tracker  
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-27-16-45-31-768163-yahboom-8600  
86  
[INFO] [launch]: Default logging verbosity is set to INFO  
[INFO] [micro_ros_agent-1]: process started with pid [860088]  
[INFO] [robot_state_publisher-2]: process started with pid [860090]  
[INFO] [joint_state_publisher-3]: process started with pid [860092]  
[INFO] [base_driver-4]: process started with pid [860094]  
[INFO] [ekf_node-5]: process started with pid [860096]  
[INFO] [laser_Tracker-6]: process started with pid [860098]  
[micro_ros_agent-1] [1785141931.885703] info      | UDPv4AgentLinux.cpp | init          | running...  
| port: 8090  
[laser_Tracker-6] [INFO] [1785141932.452721679] [laser_Tracker]: Tracker initialized. Use "ros2 param set /laser  
yahboom@yahboom: ~ 112x12  
yahboom@yahboom:~$ ros2 param set /laser_Tracker enable true  
Set parameter successful  
yahboom@yahboom:~$
```

- The robot automatically recognizes a moving target ahead and keeps following it; when the target turns, the robot automatically adjusts its direction to maintain both heading and distance
- Place an object near the robot so that its height can be scanned by the LiDAR, then run the following command in the terminal to enable tracking

```
ros2 param set /laser_Tracker enable true
```

- To stop tracking, disable tracking or simply shut down the program

```
ros2 param set /laser_Tracker enable false
```

3. Source Code Analysis

Source code path:

```
$HOME/microros_ws/src/common_demo/common_demo/lidar/laser_Tracker.py
```

Core logic of the tracking node: **enable control** → **nearest obstacle detection** → **PID closed-loop following**.

1. **Enable gate:** The `registerScan` callback first checks the `is_enabled` parameter (toggled dynamically via `ros2 param set /laser_Tracker enable true/false`). If not enabled, it returns immediately and the robot stays still.
2. **TF setup and frame identification:** Same as the obstacle avoidance node. On the first `LaserScan` received, it auto-detects the LiDAR `frame_id` and establishes the `base_link` ← `laser_frame` transform chain via `TransformListener`, ensuring that subsequent point clouds can be correctly converted to the robot body frame.
3. **Nearest obstacle detection:**
 - Iterate over all laser points and filter out invalid values (0 or beyond `range_max`)
 - Convert each point from polar to Cartesian coordinates and transform it to the `base_link` frame
 - Keep only front points whose angle is within $\pm$ `LaserAngle`, recording their distance and coordinates
 - Select the closest obstacle (`minDist`, `min_angle`) from these points
4. **PID closed-loop control:**
 - **Distance deadzone:** When `|minDist - ResponseDist| < 0.1m`, the target distance is considered reached and the linear velocity is set to 0
 - **Linear velocity PID:** With `ResponseDist` as the setpoint and `minDist` as the feedback, `lin_pid` computes the forward speed; the farther the distance, the faster the speed
 - **Angular velocity PID:** With 0 (alignment) as the setpoint and `min_angle` (in radians) as the feedback, `ang_pid` computes the steering speed
 - **Angular deadzone:** When `|ang_pid_output| < angular_deadzone`, output 0 to avoid wheel jitter caused by tiny deviations
 - **Angular clamp:** When exceeding `max_angular`, clamp to $\pm$ `max_angular` to prevent overly aggressive steering
5. **Logging and publishing:** Tracking status (distance, angle, linear/angular velocity) is logged throttled by `log_interval`, and finally the `Twist` is published to `/cmd_vel`.

```
def registerScan(self, scan_data):
    if not isinstance(scan_data, LaserScan): return

    # Skip processing if not enabled
    if not self.is_enabled:
        return

    # Auto-detect laser frame_id
    if not self.laser_frame:
        self.laser_frame = scan_data.header.frame_id
        self.get_logger().info(f'Detected laser frame: {self.laser_frame}')
```

```

# If TF is not ready, try to establish connection
if not self.tf_ready:
    try:
        self.tf_buffer.lookup_transform(
            'base_link',
            scan_data.header.frame_id,
            rclpy.time.Time(),
            timeout=rclpy.duration.Duration(seconds=0.1)
        )
        self.tf_ready = True
        self.get_logger().info(f'TF ready: base_link <-
{scan_data.header.frame_id}')
    except TransformException:
        # TF not ready, skip this scan
        return

# Get transform from laser to base_link
try:
    trans = self.tf_buffer.lookup_transform(
        'base_link',
        scan_data.header.frame_id,
        rclpy.time.Time(),
        timeout=rclpy.duration.Duration(seconds=0.1)
    )
except TransformException:
    # Throttle TF error logs
    current_time = self.get_clock().now().nanoseconds / 1e9
    if (current_time - self.last_tf_log_time) >= self.tf_log_interval:
        self.get_logger().warn('TF not available, skipping scan')
        self.last_tf_log_time = current_time
    return

# Iterate through all laser points and find closest obstacle in front
ranges = np.array(scan_data.ranges)
minDistList = []
minDistBaseList = [] # Store positions in base_link frame

for i in range(len(ranges)):
    if ranges[i] == 0.0 or ranges[i] > scan_data.range_max:
        continue

    # Calculate laser point position in laser coordinate frame
    angle = scan_data.angle_min + scan_data.angle_increment * i
    point_in_laser = PointStamped()
    point_in_laser.header = scan_data.header
    point_in_laser.point.x = ranges[i] * math.cos(angle)
    point_in_laser.point.y = ranges[i] * math.sin(angle)
    point_in_laser.point.z = 0.0

    # Transform to base_link coordinate frame
    try:
        point_in_base = do_transform_point(point_in_laser, trans)
        x = point_in_base.point.x
        y = point_in_base.point.y

        # Only consider obstacles in front (within LaserAngle)
        dist = math.sqrt(x**2 + y**2)

```

```

        angle_to_base = math.atan2(y, x) * RAD2DEG

        if abs(angle_to_base) < self.LaserAngle:
            minDistList.append(dist)
            minDistBaseList.append((x, y, angle_to_base))
        except TransformException:
            continue

    if len(minDistList) == 0:
        return

    # Find the closest obstacle
    minDist = min(minDistList)
    minIdx = minDistList.index(minDist)
    min_x, min_y, min_angle = minDistBaseList[minIdx]

    # PID control for tracking
    velocity = Twist()

    # Distance deadzone
    if abs(minDist - self.ResponseDist) < 0.1:
        minDist = self.ResponseDist

    # Linear velocity PID (maintain target distance)
    velocity.linear.x = -self.lin_pid.pid_compute(self.ResponseDist,
minDist)

    # Angular velocity PID (align with obstacle)
    # Use angle directly in radians for better sensitivity
    angle_rad = math.radians(min_angle)
    ang_pid_compute = self.ang_pid.pid_compute(0, angle_rad) # Target is 0
(aligned), current is angle_rad

    velocity.angular.z = -ang_pid_compute

    # Reduced deadzone for better responsiveness
    if abs(ang_pid_compute) < self.angular_deadzone:
        velocity.angular.z = 0.0

    # Clamp angular velocity to safe limits
    if abs(velocity.angular.z) > self.max_angular:
        velocity.angular.z = self.max_angular * (1.0 if velocity.angular.z >
0 else -1.0)

    # Throttle logging
    current_time = self.get_clock().now().nanoseconds / 1e9
    if (current_time - self.last_log_time) >= self.log_interval:
        self.get_logger().info(
            f'Tracking: dist={minDist:.2f}m, angle={min_angle:.1f}deg, '
            f'linear={velocity.linear.x:.2f}, angular=
{velocity.angular.z:.2f}'
        )
        self.last_log_time = current_time

    self.pub_vel.publish(velocity)

```

- Parameter file path

```
$HOME/microros_ws/src/microros_v2_bringup/config/common.yaml
```

Parameter descriptions for the `laser_Tracker` node in `common.yaml`:

Parameter	Default value	Description	Tuning suggestions
<code>linear</code>	0.15 m/s	Maximum forward linear velocity	Upper limit of the following speed; raising it speeds up tracking but may cause overshoot
<code>angular</code>	1.0 rad/s	Maximum steering angular velocity	Upper limit of the steering response speed; too large causes oscillation
<code>LaserAngle</code>	45.0°	Half-angle of the target search sector	The larger the angle, the wider the search range, but lateral interference may be introduced
<code>ResponseDist</code>	0.4 m	Following hold distance	The desired distance to keep from the target; adjust according to the scene
<code>enable</code>	false	Whether following is enabled at startup	Disabled by default; enable manually with <code>ros2 param set</code>
<code>lin_pid_P/I/D</code>	1.0/0.0/1.0	Linear velocity PID parameters	P determines response speed, D suppresses oscillation, I eliminates steady-state error
<code>ang_pid_P/I/D</code>	2.0/0.0/2.0	Angular velocity PID parameters	P determines steering response, D suppresses overshoot; recommended to start with P=D
<code>angular_deadzone</code>	0.02 rad/s	Angular velocity deadzone	Below this value no steering is output, avoiding jitter from tiny deviations
<code>max_angular</code>	2.0 rad/s	Angular velocity limit	Limits the maximum steering speed to prevent overly aggressive turns

```
laser_Tracker:
  ros__parameters:
    linear: 0.15          # Maximum forward linear speed (m/s)
    angular: 1.0          # Maximum steering angular velocity (rad/s)
    LaserAngle: 45.0      # Half-angle of the target search sector
                          (deg)
    ResponseDist: 0.4      # Following hold distance (m)
    enable: false          # whether following is enabled at startup
```

```
# ---- PID ----
lin_pid_P: 1.0          # Linear velocity PID: proportional
lin_pid_I: 0.0          # Linear velocity PID: integral
lin_pid_D: 1.0          # Linear velocity PID: derivative
ang_pid_P: 2.0          # Angular velocity PID: proportional
ang_pid_I: 0.0          # Angular velocity PID: integral
ang_pid_D: 2.0          # Angular velocity PID: derivative

# ---- Angular velocity deadzone and limiting ----
angular_deadzone: 0.02  # Angular velocity deadzone (rad/s), wheels
do not turn below this value
max_angular: 2.0        # Angular velocity limit (rad/s)
...
```